

Self-Hosted LLM Observability — Findings Report

Environment: NVIDIA RTX 1000 Ada (6 GB) / Windows 11 · Ollama → Qwen 2.5 7B Q4_K_M · Dynatrace qof78400 **EC2 (extended testing):** A10G (24,576 MiB) · Amazon Linux 2023 · ec2-user@13.220.30.235

1. GPU Observability Stack

Two NVML shippers were evaluated. The Python shipper supersedes the Node.js one.

Component	File	Metrics	Mechanism
Node.js GPU Shipper	gpu-metrics.js	10	nvidia-smi CLI polling
NVML GPU Shipper	gpu-metrics-nvml.py	32	pynvml direct NVML C-API, 2 s interval
OneAgent LLM Proxy	serve.js	spans + headers	OTel OTLP → DT Grail
Strato GPU Dashboard	app-gpu/	—	DQL queries on Grail metrics

The NVML shipper exposes metrics unavailable via nvidia-smi including: PCIe throughput, hardware energy counter (gpu.energy.delta_joules), P-state, all throttle reason flags, and encoder/decoder utilization. See skill: gpu-metrics-shipper.

2. VRAM Event Detection — Fix and Classification

Problem

The original detector used a single -500 MiB threshold for all VRAM drop events, causing model unloads (-18,000 MiB) to be misclassified as KV cache compaction.

Fix: Magnitude-Based 3-Tier Classification

```
if delta_mib < -10_000:
    event_type = "model_unload_event"      # model fully evicted from GPU
elif delta_mib < -500:
    event_type = "vram_eviction_event"     # OOM recovery / partial unload
elif delta_mib < -50:
    event_type = "kv_cache_compaction_event" # context-cap truncation
```

Empirical basis:

- Model unload: RTX 1000 Ada loses ~5,800 MiB (Q4_K_M weights) in one step → always > 10,000 MiB on A10G (qwen3:32b ~20,000 MiB)

- KV compaction: Ollama truncates the KV cache to recover context headroom — measured at -221 MiB on A10G with qwen3:8b at 80K tokens
- Eviction: Intermediate range covers partial unloads and OOM recovery under multi-model pressure

Status: Fixed, deployed to EC2, confirmed active via systemd.

Reference skill: [gpu-vram-kv-cache-signals](#)

3. KV Cache VRAM Scaling

For qwen3:8b (28 layers, 4 KV heads, 128 head_dim, FP16 KV):

$$\text{kv_cache_bytes} = 2 \times 28 \times 4 \times 128 \times \text{num_ctx} \times 2 \approx 0.143 \text{ MiB / token of context}$$

Hard context cap: 131,072 tokens → ~18,700 MiB KV at full utilization (exceeds A10G headroom above model weights).

Practical cap: VRAM floor is occupied by model weights (~5,000 MiB on A10G) leaving ~19,500 MiB for KV. Observable as a VRAM utilization plateau when context-cap kicks in — the model stops growing and Ollama starts truncating.

Context cap discovery pattern: Send progressively longer prompts; plot `gpu.memory.used_mib` vs prompt tokens. The slope breaks when the software cap is reached.

Reference skill: [gpu-vram-kv-cache-signals](#)

4. TTFT Linearity as a Health Signal

Time-to-first-token scales linearly with prompt token count on healthy hardware:

$$\text{TTFT}(n) = \text{slope} \times n + \text{intercept}$$

R² from OLS regression over recent requests is a real-time health indicator:

R ² Range	Interpretation
0.95 – 1.00	Healthy — GPU operating nominally
0.85 – 0.95	Mild degradation (thermal ramp-up, memory pressure)
0.70 – 0.85	Significant issue — investigate throttle flags
< 0.70	Severe — likely thermal throttling or KV compaction artifacts

Measured on A10G (qwen3:8b): slope ≈ 0.8 ms/token, intercept ≈ 120 ms.

Deviations from predicted TTFT indicate: thermal throttling, KV cache compaction events, memory bandwidth saturation, or hardware degradation. Ship `gen_ai.ttft_ms` and `gen_ai.request.prompt_tokens` per request to compute this in DQL.

Reference skill: `llm-ttft-linearity-health`

5. Energy Cost per Inference

NVML hardware counter (`nvmlDeviceGetTotalEnergyConsumption`) returns cumulative millijoules from the GPU die — significantly more accurate than `power_draw × elapsed_time` due to sub-millisecond power fluctuations during inference.

Measured baseline (qwen3:8b, A10G):

Phase	Energy
Prefill (per 1K prompt tokens)	~0.8 J
Decode (per completion token)	~0.04 J
Typical 40-token prompt + 420-token completion	~0.51 J total

Energy scaling under KV pressure: As context grows toward the software cap, decode cost per token rises because memory bandwidth becomes the bottleneck (larger KV slices read per step). A 2–3× decode energy increase at 80%+ context utilization is normal.

Cost estimation: At US commercial GPU rates (~\$0.50/GPU-hour for A10G), and 400 J/hr sustained inference: ~\$0.0001 per typical completion. Negligible — thermal limits and throughput capacity matter more.

Reference skill: `llm-energy-cost-analysis`

6. Temperature Sweep (Qwen 2.5 7B, RTX 1000 Ada)

Prompt: "What are fun things to do in Reynolds, IL?" — 2 runs per temperature, 40 prompt tokens.

Finding: No statistically significant correlation between temperature (0.0–2.0) and output token count at this sample size (avg ~420 completion tokens). Quality degrades visibly above 1.5 (incoherent outputs) but token count stays flat. **Use temperature 0.3–0.7 for production** — low enough for consistency, high enough to avoid degenerate repetition.

7. Context Persistence Middleware

Problem: Self-hosted LLMs have hard context caps (131,072 tokens). Clients sending long conversations hit the cap silently — the model starts forgetting earliest turns. GPU signals arrive *after* the eviction (lagging indicators), not before it.

Solution: Zero-config proxy middleware (`context-manager.js`) embedded in `serve.js`.

Architecture

```
Client → serve.js → contextManagerMiddleware → Ollama
      |
      | SessionManager (per X-Session-Id)
      | HealthMonitor (ollama /api/ps, 5s poll)
      | NVML Event Reader (nvml-events.jsonl, 2s)
      | ContextPersister (summarize + write MEMORY.md)
```

Dynamic Threshold

```
effective_threshold = floor(CONTEXT_CAP × PERSIST_RATIO × gpu_multiplier)
```

GPU Signal	Multiplier Adjustment
Baseline (healthy)	1.0 (persist at 75% of cap)
VRAM util ≥ 80% (sustained)	−0.05 per poll cycle, min 0.6
VRAM util ≥ 88% (critical)	−0.10 per poll cycle, min 0.5
<code>kv_cache_compaction_event</code>	−0.15, min 0.5
<code>vram_eviction_event</code>	−0.20, min 0.4
<code>model_unload_event</code>	0.5 + emergency dump all sessions
5 min healthy window	+0.02 per poll (relaxes back to 1.0)

Persistence Behavior

When threshold is crossed before forwarding a request to Ollama:

1. Oldest 60% of turns are summarized via a small LLM call (`num_ctx: 8192`, `temp: 0.3`)
2. Summary preamble replaces those turns in the session
3. Full conversation appended to `sessions/{id}/MEMORY.md` with trigger metadata
4. Trimmed `messages[]` substituted into `req.body` before Ollama sees the request
5. Client receives a normal response — no indication anything was swapped

Fallback: If Ollama is unreachable for summarization, an extractive summary (first + last line per turn) is used instead.

Test Results

15 tests – 15 passed, 0 failed

- ✓ `estimateTokens` with empty array returns 0
- ✓ `estimateTokens` counts tokens roughly correctly
- ✓ `estimateTokens` scales linearly with message count
- ✓ `getOrCreateSession` creates new session

- ✓ `getOrCreateSession` returns existing session
- ✓ `gpuHealth` starts at healthy defaults
- ✓ `threshold multiplier` adjusts effective threshold
- ✓ `middleware` skips non-chat requests
- ✓ `middleware` skips requests without messages
- ✓ `middleware` calls next for below-threshold requests
- ✓ `middleware` triggers persistence when above threshold (mock summarize)
- ✓ NVML compaction event tightens threshold multiplier
- ✓ NVML model unload event drops multiplier to minimum
- ✓ `health endpoint` returns correct structure
- ✓ NVML event file processing (end-to-end)

Observed: 60-message test conversation (1,800 estimated tokens) triggered persistence at threshold 1,500. 36 turns summarized → session trimmed to 25 messages. `MEMORY.md` written with checkpoint header, trigger metadata (token count, threshold, VRAM %, multiplier).

Reference skills: `gpu-vram-kv-cache-signals`, `self-hosted-llm-ollama-qwen`

8. Skills Generated This Session

Six agentic skills were created capturing all domain knowledge from this investigation. All are available globally in `c:\Users\josh.wood\.agents\skills\`.

Skill	Purpose
<code>self-hosted-llm-ollama-qwen</code>	Ollama + Qwen setup, quantization, <code>num_ctx</code> behavior, software context caps, VRAM estimation, keep-alive tuning
<code>llm-observability-instrumentation</code>	Full DT telemetry stack: OTel spans with <code>gen_ai.*</code> attributes, MINT metrics, context sweep automation, TTFT/throughput shipping
<code>gpu-vram-kv-cache-signals</code>	VRAM event classification (3-tier magnitudes), KV cache scaling math, utilization as capacity signal, compaction detection patterns
<code>gpu-cupti-kernel-telemetry</code>	CUPTI LD_PRELOAD injection into Ollama subprocesses, Activity API for kernel energy/memory events, subprocess discrimination
<code>llm-energy-cost-analysis</code>	NVML hardware energy counters, J/tok methodology, prefill vs decode decomposition, energy scaling under KV pressure
<code>llm-ttft-linearity-health</code>	R^2 as GPU health indicator, OLS regression, deviation alerting, breakdown scenarios (throttle, compaction, bandwidth saturation)

9. Files Changed This Session

File	Change
<code>gpu-metrics-nvml.py</code>	VRAM event detector: single -500 threshold → 3-tier magnitude classification; deployed to EC2

File	Change
<code>serve.js</code>	Added <code>contextManagerMiddleware</code> + <code>/v1/context-health</code> health endpoint
<code>context-manager.js</code>	New — session tracking, GPU health monitor, NVML event reader, persistence logic
<code>test-context-manager.js</code>	New — 15-test integration suite for <code>context-manager.js</code>

10. Open Items

Item	Notes
NVML event file writer	<code>gpu-metrics-nvml.py</code> currently ships events to DT only; needs local <code>nvml-events.jsonl</code> writer so <code>context-manager.js</code> can consume compaction events without polling DT
Session ID propagation	Clients must send <code>X-Session-Id</code> header for accurate session tracking; without it, sessions are keyed by first-message hash (breaks on reconnect)
EC2 deployment	<code>context-manager.js</code> not yet pushed to EC2 / wired into EC2 <code>serve.js</code>
Multi-model sessions	<code>CONTEXT_CAP</code> is global; sessions using different models need per-model caps